

Lab 2 - Roofline Analysis

Advanced Computer Architectures

University of Minho

Lab 2 focus on the creation of a Roofline model of a server for a sample application using the Intel Advisor software suite. Use a cluster node (through `sbatch` script submission) to run the analysis using the command line.

This lab tutorial includes three exercises, one to be solved at home (HW 1.1) and three to be solved during the lab class (Lab 2.x).

The goal of Lab 2 is to evaluate the maximum computing throughput and memory bandwidth of a computing node, while identifying the relevant ceilings that may impact performance of a sample application. The performance data gathered from an application allows it to be plotted into a Roofline graph for a given node, in order to assess its optimisation efficiency (how far it is from a theoretical maximum). The relative position of an application arithmetic intensity relatively to the computing node roof and ceilings gives an insight into the inefficiencies of the code, whose limitations can be better identified and optimised.

To load the compiler in the environment use these commands (use these or newer versions of each software):

GNU Compiler (for several system libraries): `module load gcc/11.2.0`

Intel Advisor: `module load oneapi/advisor/latest`

Consider the code available in the `/projects/F202500001HPCVLABEPICTURE/CHED/lab2` folder in the Deucalion supercomputer. There are 3 different groups of algorithms, each with a different arithmetic intensity, and thus different bottlenecks. The algorithms to test must be chosen before compiling the code by (un)commenting the appropriate `define` in the `src/roofline.h` file.

1. Installing and using the Intel Advisor

Goals: to get acquainted with the Intel Advisor interface and the Roofline model.

HW 1.1 Deucalion provides two alternatives to visualise the profiling output from Intel Advisor CLI measurements: through a desktop session with OpenOnDemand (use `advisor-gui` in the desktop's terminal); by creating interactable html pages (only works for Roofline visualisation).

Alternatively, download the most recent Intel Advisor and install it on your laptop, or install a Docker image. Ideally, the same version must be used on the cluster and on your laptop. This installation can be used to visualise the Roofline created for the computing node, but also other analyses such as vectorisation efficiency and memory access patterns.

2. Analysing Rooflines with Intel Advisor

Lab 2.1 Access Deucalion and copy folder for this lab session into your home directory. Enable all algorithms in Group 1 in the code and compile it using the **Makefile** provided.

Submit a job to profile the code and create the Roofline model containing the dots with the adequate arithmetic intensity for the compiled application. You may need to modify the provided job script:

```
advisor --collect survey --enable-cache-simulation --project-dir ./roofline
-- ./bin/nbody options
advisor --collect tripcounts --project-dir ./roofline --flop --no-trip-counts
-- ./bin/nbody options
```

Copy the **roofline** folder to your laptop and open the project file with your own installation of the Intel Advisor. A Roofline plot should be displayed inside a window of Intel Advisor. Alternatively, generate the html and copy it over to your laptop for visualisation. It can be generated using the following command:

```
advisor --report=roofline --project-dir=./roofline --report-output=./roofline.html
```

Answer the following questions:

1. What does the **survey** analysis do?
2. What represent the size and colour of each dot in the Roofline chart dot?
3. What represent the diagonal lines in the Roofline chart?
4. What represent the horizontal lines in the Roofline chart? Where should be positioned a dot representing a loop that is not vectorized?
5. Where should be positioned a dot that offers a better opportunity for performance improvement?
6. Where are the limiting ceilings for the different data structures (AoS, SoA, and SoA with vectorization) used by the algorithms?

Lab 2.2 The compiled code executes several nbody simulations with different implementations (and optimisations).

Based on the data gathered by Advisor for these algorithms answer the following questions:

1. Does the displayed Roofline show the peak performance of this cluster node?
2. Which displayed ceilings are not relevant to assess the performance limitations of these algorithms? Remove them from the Roofline visualisation.
3. What is the key optimisation on `nbody_step_v4` that is not present in `nbody_step_v1`? Does this optimisation allow `nbody_step_v1` to break a ceiling limiting the performance of `nbody_step_v4`?

4. What is the current limitation of the `nbody_step_v5` algorithm? Is it possible to find clues in Intel Advisor that point to this limitation?
5. Can the performance of `nbody_step_v5` be further improved? What would be the maximum theoretical performance (in GFLOP/s) that this code could achieve?
6. What is the ceiling limiting the performance of `nbody_step_v2`? Is it the same as `nbody_step_v4`? Does Intel Advisor provide clues to support your claims?
7. What is the vectorization efficiency of `nbody_step_v5`? Why is this value so high/low?
8. Explain the key optimisation implemented on the fastest algorithm. How does this optimisation affect the code execution?
9. How much slower is the best performing algorithm relatively to the peak theoretical performance?

Lab/Ext 2.3 These implementations do not take full advantage vectorisation. Identify one algorithm that may benefit the most from such optimisation. Modify the `Makefile` to use the most advanced vector instruction set available in the Ivybridge architecture, create the Roofline and compare it to the original version of the code you chose. Check if the Roofline ceilings are adequately configured to consider the use of multiple CPU cores.